

Engineering Onboarding Copilot

New engineers lose days discovering conventions.

Reviewers lose hours correcting avoidable issues.

PM, QA, and DevOps lack visibility.

This tool eliminates that tax.

Architecture: Two Layers, One Profile

Cursor Workspace (IDE layer)

Rules + MCP Server

↕ reads from ↕

profiles/scikit-image.yaml

↕ reads from ↕

CLI Engine (ob)

scaffold · check · brief

↓ same checks ↓

GitHub Actions CI Pipeline

Team owns the YAML. Engine owns the logic.

Built / Deferred / Why

Built	Deferred	Why
ob check — 5 rule types	Semantic code review	Deterministic first
ob scaffold — guardrails	Multi-file refactors	Scope
ob brief — 4 roles	Code-aware briefs	Template > LLM
Cursor rules + MCP	Auto-convention discovery	Manual curation = honest
CI pipeline	Cross-repo profiles	Single-library scope
Diffusers stub profile	Full diffusers support	Proves extensibility

Where It Breaks

- **Deterministic only** — no semantic understanding
- **Single profile** — one library at a time
- **Template briefs** — metadata-driven, not code-aware
- **Hand-curated** — no automated convention discovery
- **No multi-file refactoring** support

Each is a deliberate choice. Each has a remediation path.

Extensibility → Round 4

Swap the YAML. Keep the engine.

`profiles/diffusers.yaml` proves the architecture:

- New profile → new conventions enforced
- Same CLI, same rules, same MCP
- One file change, three surfaces updated

A new customer library = a new YAML, not a new tool.

The Close

“ Most candidates build a coding assistant for one engineer.
I built a **convention-aware SDLC workflow** that turns tribal knowledge into an executable artifact — owned by the team, usable by PM, QA, and DevOps, and extensible to any library by swapping a YAML profile. ”